Atividade prática 4 - Buffer Overflow

Universidade Federal do ABC (UFABC)
Segurança de dados (turmas NA1MCTA023-17SA e NA2MCTA023-17SA)
Primeiro Quadrimestre de 2026
Professora: Denise Goya

Sobre a atividade

Esta atividade tem o objetivo de explorar conceitos de Programação Segura e de Segurança no Sistema Operacional. Especificamente, você experimentará alguns aspectos relacionados a **buffer overflow** na pilha de execução dos processos do SO (*stack*), também conhecido por stack overflow.

Você deverá conduzir a atividade de forma independente (individual ou em dupla), tirando dúvidas com o professor caso seja necessário.

Pré-requisitos

Esta tarefa deve ser feita no Linux da **máquina virtual do SeedLabs**. Não é recomendável realizá-la em máquinas reais de trabalho Linux, pois medidas protetivas do SO e compilador serão desativadas ao longo da atividade.

Entrega e Avaliação

Para avaliação, responda individualmente ao **questionário** disponível no Moodle. Trata-se de um teste com correção automática. O prazo para entrega é de uma semana após a aula de laboratório. Atente-se ao prazo, pois, uma vez encerradas as submissões, o questionário não pode ser reaberto. As questões em azul podem ser respondidas em seu caderno, para apoiar seus estudos.

Completar a Tarefa 3 com sucesso e compreendê-la integralmente pode ser mais desafiador. Para avaliar a conclusão desses dois momentos (completar e compreender), você pode redigir e entregar um **Relatório** sobre a realização da **Tarefa 3**, que contará bônus para o fechamento dos conceitos (não será prejudicado quem não entregar o relatório, mas o questionário é obrigatório para todos).

Sobre a vulnerabilidade Buffer Overflow e preparo da VM

O transbordo de memória (buffer overflow) ocorre quando um programa preenche uma área de memória (buffer, vetor ou array) além do espaço previamente reservado para ela. No arquivo `Lab4.zip`, a função `func` do programa `vulneravel1.c` exibe um exemplo de código com vulnerabilidade para buffer overflow.

Sistema Operacional, processadores e compiladores oferecem algumas medidas de proteção contra buffer overflow. Precisaremos desativar algumas delas, para que seja possível explorar a vulnerabilidade de forma mais fácil.

Para iniciar, abra um terminal na **VM SeedLabs** e execute os comandos em verde a seguir, respectivamente para desabilitar a aleatorização de endereços na pilha de execução pelo SO e para linkar um shell de linha de comando:

```
$ sudo sysctl -w kernel.randomize_va_space=0
$ sudo ln -sf /bin/zsh /bin/sh
```

Execute os comandos acima sempre que reiniciar a VM para executar as tarefas desse laboratório.

Tarefa 0

Descompacte os arquivos de `Lab4.zip` para uma pasta na **VM**. Compreenda o que faz o código `vulneravel1.c` e como ele deve funcionar. Em um terminal, execute os comandos a seguir, respectivamente para (1) compilar o programa `vulneravel1.c` sem medidas de proteção contra buffer overflow e com opções que facilitam a exploração e (2) para executá-lo com strings de entrada variadas (uma válida e algumas inconsistentes com o esperado) e observe os resultados:

```
$ gcc -m32 -O0 -ggdb -static -U_FORTIFY_SOURCE -fno-stack-protector
-z execstack -no-pie -o vulneravel1 vulneravel1.c
$ ./vulneravel1 AAAA
$ ./vulneravel1 AAAABBBBCCCC
$ ./vulneravel1 AAAABBBBCCCCDDDD
```

Caso a última execução não tenha gerado falha de segmentação, experimente aumentar a string de entrada.

Perguntas da Tarefa 0

1) Explique porque o programa vulneravel1.c é passível de buffer overflow.
2) Para entradas de quantos bytes o programa retorna ao main?
3) Para entradas de quantos bytes o programa gera falha de segmentação (e, portanto, aborta a execução sem retornar ao main)?

Analisando a memória e a pilha de execução com o GDB

Para melhor compreender o que ocorreu durante a execução do programa `vulneravel1` sob as variadas entradas sugeridas, vamos usar o depurador GDB e rastrear o conteúdo da memória. Inicie a depuração do programa, usando os comandos em verde a seguir:

```
$ gdb vulneravel1
gdb-peda$ set args "AAAA"    > inicializa a string
gdb-peda$ b 7                > Breakpoint 1 at vulneravel1.c, line 7.
gdb-peda$ run                > executa até o 1º breakpoint (antes de iniciar a variável c)
```

Neste ponto, observe quais são os parâmetros passados para `func` (estão ok).

Adicione outro breakpoint na linha 10, continue a execução e confira o conteúdo das variáveis `c` e `str` (estão ok):

```
gdb-peda$ b 10
gdb-peda$ continue     > continua a execução até o próximo breakpoint
gdb-peda$ p c          > exibe o conteúdo da variável c
gdb-peda$ p str        > exibe o conteúdo da variável str
gdb-peda$ continue     > continua a execução até o próximo breakpoint (no caso, encerra)
```

Se o programa voltou ao main e finalizou como esperado, você fez tudo corretamente (caso não, refaça a depuração do início).

Com o GDB em ação, vamos reiniciar a execução do programa, mas agora forçando um overflow:

```
gdb-peda$ set args "AAAABBBBCCCC"      > inicializa a string
gdb-peda$ run          > executa até o 1º breakpoint (antes de iniciar a variável c)
gdb-peda$ b 9          > Breakpoint antes de fazer a cópia da string
gdb-peda$ continue     > continua a execução até o próximo breakpoint (l. 9)
gdb-peda$ p c          > exibe o conteúdo da variável c
gdb-peda$ continue     > continua a execução até o próximo breakpoint (l. 10)
gdb-peda$ p c          > exibe o conteúdo da variável c
```

Se o valor de c mudou de 0xdeadbeef para 0x42424242, você está observando o overflow (de “BBBB” sobre c, pois 0x42=“B”). Refaça do início, se algo deu errado.

Neste ponto, ainda antes de finalizar `func`, vamos observar o registrador EBP, que guarda um ponteiro de base da função em execução na pilha. No exemplo da máquina a seguir, o EBP vale `0×ffffd158`. Esse valor mais 4 (=`0×ffffd15c`) contém o endereço de memória que guarda o endereço de retorno da função `func` para a função `main`.

```
EBP: 0xffffd158

0024| 0xffffd158 --> 0xffffd178 --> 0x0
0028| 0xffffd15c --> 0x8049e38 (<main+59>:
```

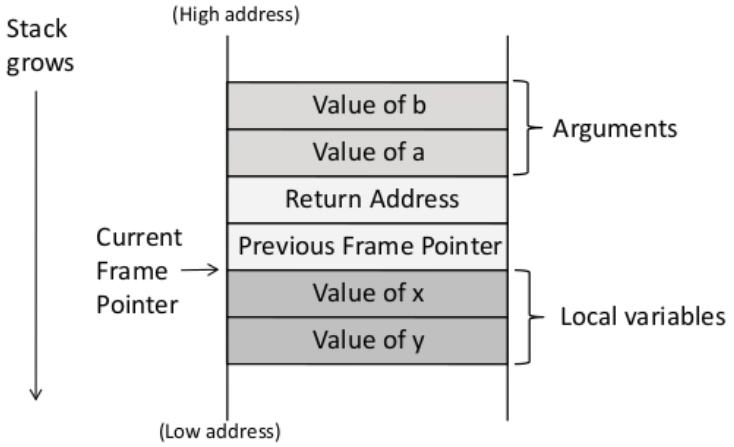
Descubra quais são o EBP e onde está o endereço de retorno de `func` em sua máquina, com:

`gdb-peda$ p $ebp` > exibe valor de EBP (ponteiro base da função em execução)
`gdb-peda$ p $ebp+4` > exibe EBP+4 (endereço que contém o endereço de retorno da função em execução)

Com o GDB é possível, por exemplo, alterar o conteúdo de qualquer localidade da memória e, se compilado com a opção `-ggdb` do gcc, é possível usar nomes de identificadores (alto nível) de variáveis e de funções durante a depuração. Por exemplo, com os comandos a seguir, é possível obter o endereço de onde se inicia a função `outraFuncao` e atualizar o endereço de retorno da função em execução na pilha:

`gdb-peda$ p &outraFuncao` > exibe o endereço de outraFuncao
`gdb-peda$ set {int}($ebp+4)=0×babaca0f` > altera o retorno da função

Vimos na aula de teoria que uma função em execução tem seus dados armazenados na pilha, conforme a figura a seguir em que, a função possui como parâmetros os valores (a, b) e declara as variáveis locais (x, y). O registrador EBP guarda o “*current frame pointer*”. Como o programa `vulneravel1` foi compilado para arquitetura de 32 bits (opção `-m32` do gcc) cada palavra (isto é, cada item da figura) possui 4 bytes (assim, ebp+4 aponta para onde está o endereço de retorno).



Se você respondeu as questões 2 e 3 da Tarefa 0, mas não compreendeu muito bem a razão por trás das respostas, procure mapear as variáveis e parâmetros de `func` na figura acima e calcule se a quantidade de bytes estava dentro do esperado. Experimente o comando abaixo para exibir a diferença entre a posição do endereço de retorno e o início da variável local `buf` (resultado em hexadecimal). Lembre que o preenchimento do buffer ocorre de baixo para cima (ao contrário do crescimento da pilha, que é de cima para baixo), razão pela qual o overflow de `buf` pode comprometer o comportamento do programa de muitas formas (desde os bugs mais triviais como *segmentation fault*) até outras mais sofisticadas, como as das tarefas seguintes. O exemplo a seguir mostra o resultado de uma conta com endereços de memória. Para interpretar o resultado, é preciso compreender o contexto do depurador e lembrar que contas em hexadecimal produzem resposta em hexadecimal.

```
gdb-peda$ p $ebp+4-&buf[0]
```
> exibe o resultado do cálculo (aritmética hexadecimal)

O comando **`quit`** encerra o GDB.

Tarefa 1

Use o GDB para forçar a execução das funções `outraFuncao` e `bar`, que não estavam sendo chamadas no programa original. A função `bar` faz uma chamada de sistema e executa um shell, que dá acesso a uma linha de comando, com os mesmos privilégios de acesso do dono do programa original. Não avance para os próximos parágrafos caso não tenha obtido sucesso. Consulte o professor ou colegas para superar qualquer dificuldade.

Sob a linha de comando que você conquistou dentro do GDB, experimente alguns comandos Linux como:

- `ls -al`
- navegue entre diretórios
- `apt-get install bolinha`

Provavelmente os dois primeiros exemplos acima foram bem sucedidos. No terceiro caso, você deve ter tido "permissão negada", pois é necessário privilégio de root para tanto.

Encerre o GDB com **`quit`**. E repita o experimento de forçar a execução da função `bar`, de dentro do GBD, mas agora com privilégios de superusuário, digitando:

```
$ sudo gdb vulneravel1
```

Explique o resultado dessa nova tentativa sob a linha de comando conquistada:

- `apt-get install bolinha`

O que fizemos até agora foi apenas um aquecimento para um próximo passo em direção a um ataque de buffer overflow. A rigor, não é esperado que um atacante tenha acesso a um executável compilado com a opção `-ggdb` (que produz informações úteis para o depurador GDB). Ainda mais improvável é que um atacante tenha acesso a um programa vulnerável que possua uma função que dê acesso a um shell. No entanto, em muitos casos é possível provocar overflow e injetar o código de um shell na memória e desviar a execução do programa para esse código injetado.

Perguntas da Tarefa 1

4) Exiba telas do que você fez para forçar a execução das funções outraFuncao e bar dentro do GDB, explicando cada passo.
5) Explique com suas palavras porque um programa com vulnerabilidade de buffer overflow pode ser perigoso tanto para um usuário comum quanto para um computador servidor.
6) Pesquise o significado de **setuid** (set user ID upon execution) e discorra sobre o impacto que poderia causar uma exploração bem sucedida de vulnerabilidade de buffer overflow sobre um programa com privilégio de root.

Tarefa 2

O programa `vulneravel2.c` disponível em `Lab4.zip` é semelhante ao da Tarefa 1, mas lê a string a partir de um arquivo. **Seu objetivo nesta Tarefa 2** é construir um arquivo com uma string que explore a vulnerabilidade de buffer overflow de

`vulneravel2.c` e **acione a execução da função `bar`**, dando total controle a você, um suposto invasor da máquina (em ambiente mais próximo do real, isto é, de fora do GDB).

O programa `monta_string.py` disponível em `Lab4.zip` contém um exemplo que gera um arquivo de entrada que pode ser usado pelo programa `vulneravel2.c`.

Para iniciar, execute os comandos a seguir, respectivamente para (1) compilar o programa vulnerável sem medidas de proteção contra buffer overflow e com opções que facilitam a exploração e (2) criar um arquivo de entrada para exploração da vulnerabilidade:

```
$ gcc -m32 -O0 -ggdb -static -U_FORTIFY_SOURCE -fno-stack-protector
-z execstack -no-pie -o vulneravel2 vulneravel2.c
```

```
$ ./monta_string.py          (ou: python3 monta_string.py)
```

Em seguida, execute o programa `vulneravel2` com o arquivo gerado por `monta_string.py` (`badfile1`) e observe se você obtém a saída abaixo:

```
$ ./vulneravel2 badfile1
```

Tamanho da string: 12
==== Retornou ao main corretamente! ====

Use um editor hexadecimal para visualizar o arquivo `badfile1` (na VM, já está instalado o *Bless Hex Editor*) e confira o conteúdo dos bytes. Observe que o endereço de retorno foi gravado em ordem *little endian* (bytes da direita para esquerda), que é o padrão da arquitetura usada na VM e em grande parte das máquinas de outras plataformas.

Perguntas da Tarefa 2

7) Em função do que você aprendeu com as Tarefas, explique porque `badfile1` permitiu o retorno ao programa principal, mesmo tendo ocorrido overflow.
8) Exiba telas do GDB e do código que você elaborou para gerar uma string via `monta_string.py`, explicando como e porque você conseguiu provocar a chamada à função **`bar`** a partir da execução do programa `vulneravel2` (de fora do GDB). Justifique cada passo.

Tarefa 3

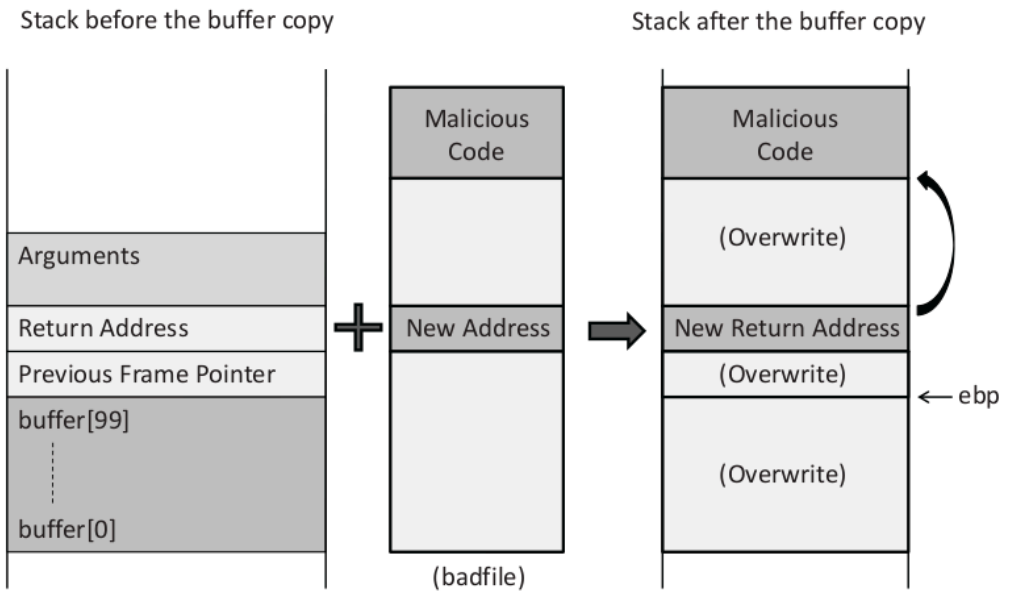
O programa `stack.c` disponível em `Lab4.zip` é semelhante ao `vulneravel2.c` da Tarefa 2. Difere essencialmente pela função `dummy_function` que não faz nada específico, mas é a responsável por chamar a função vulnerável `bof`.

Como `main` chama `dummy_function`, que chama `bof`, temos um espaço conveniente para **injetar um shellcode na pilha** via buffer overflow, sem que o SO aborte a execução do programa por acessos indevidos na pilha.

O programa `exploit.py` disponível em `Lab4.zip` contém um esqueleto de código que gera um arquivo de entrada que pode ser usado para explorar a vulnerabilidade contida no programa `stack.c`. A variável `shellcode` é inicializada com um código de máquina que aciona a execução de um shell (linha de comando), que funciona em arquitetura de 32 bits.

Seu objetivo nesta Tarefa 3 é construir um arquivo de entrada que induz o programa `stack.c` a **acionar a execução do shellcode**. Note que `stack.c` não contém nada semelhante à função `bar` e <u>você não pode recodificar</u> `stack.c` para substituir o executável da máquina-alvo de seu ataque.

A figura a seguir esquematiza como você pode cumprir a tarefa. Construa `badfile` de modo a induzir overflow via cópia de string. O endereço de retorno da função vulnerável deve ser alterado para onde está o shellcode (representado por *Malicious Code* na figura).



Detalhes adicionais para execução da Tarefa

Para compilar `stack.c` de forma a facilitar a exploração faça:

```
$ gcc -m32 -ggdb -fno-stack-protector -z execstack -o stack stack.c
```

Lembre-se de desabilitar a aleatorização da pilha e estabelecer o link para zsh caso tenha reiniciado a VM.

Se você concluiu corretamente a Tarefa 1, notou diferença quando o acesso ao shell acontece com privilégio de root. Com os comandos a seguir, executados nessa ordem, (1) root passa a ser o dono do programa stack e (2) o bit **setuid** é ligado, e a execução de shellcode também terá privilégio de root. Note que você deve executar os comandos a seguir para observar diferenças entre cenários de ataque com e sem eles, pois é relativamente usual que processos relacionados a funções básicas do SO, por exemplo para acesso a hardware e rede, efetuem tais elevações de privilégios.

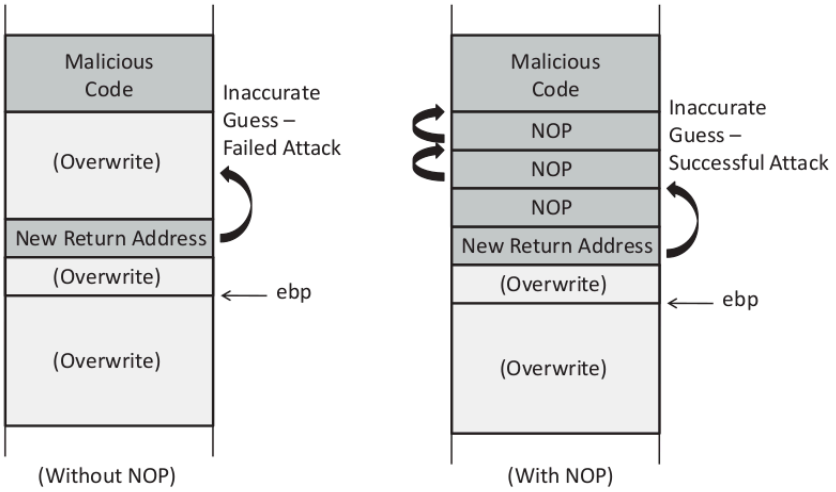
```
$ sudo chown root stack
$ sudo chmod 4755 stack
```

Você pode usar o GDB para calcular quantos bytes há entre o início do buffer e a posição que guarda o endereço de retorno (insira este valor na variável `offset` de `exploit.py`). Antes de executar o GDB, crie um arquivo `badfile` vazio (basta digitar na linha de comando:

```
$ touch badfile
```

O GDB também pode lhe informar qual é a posição de memória que guarda o endereço de retorno da função vulnerável. Use-a para atualizar a variável `ret`. Mas lembre que o ataque final acontecerá fora do GDB e ele próprio ocupa alguns bytes na pilha. Sem o GDB na memória, cerca de 0x80 pode ser somado à posição que você supõe estar o endereço de retorno alvo (tente outros valores se não funcionar de cara, pois depende de quais programas você está executando na VM).

Outra forma de facilitar o ataque é posicionar o shellcode nas últimas posições de badfile e precedê-lo de vários 0x90, instruções No-operation ou NOP. Em tempo de execução, NOP apenas avança o ponteiro PC (*program counter*) para a próxima instrução. A figura a seguir ilustra a ideia dessa estratégia.



Uma vez que você tiver concluído a edição da exploração, basta gerar o `badfile` e executar o programa vulnerável:

```
$ ./exploit.py          (ou: python3 exploit.py)
$ ./stack
```

Perguntas da Tarefa 3

9) Exiba telas do GDB e do código que você elaborou para efetivar o ataque. Justifique cada alteração realizada no código dado e cada valor usado (sem isso, o relatório não receberá pontuação máxima).
10) Relacione todas as condições que possibilitaram o ataque de buffer overflow sugerido na Tarefa 3. Para cada condição que favoreceu a exploração, indique uma contramedida que pode ser usada para dificultá-la. Seja detalhista, analise em profundidade cada passo e justifique cada afirmação.